Univerzita Karlova
Matematicko-fyzikální fakulta
Programování v jazyce C++ – NPRG041

Zápočtový program

Framework pro deskové hry

Jakub Jankovec, 2. ročník

Letní semestr 2023/24

Obsah

1. Úvod 3
1.1. Specifikace 3
1.2. Konečné řešení 3
2. Uživatelská dokumentace 4
2.1. Ovládání po spuštění 4
2.2. Specifikace framework 5
2.3. Systém událostí (event system) 6
2.4. Herní kontext (GameContext) 8
2.5. Hledání cest (pathfinding) 11
2.6. The Royal Game of Ur 11
3. Programátorská dokumentace 13
3.1. OpenGL 13
3.2. Game Manager 13
3.3. Tabletop Framework 14
4. Možná rozšíření 16
5. Závěr 16

1. Úvod

Pro splnění předmětu bylo mým úkolem vytvořit větší programové dílo v jazyce C++.

1.1. Specifikace

Cílem je vytvořit framework, který umožní uživateli vytvořit deskovou hru typu *Člověče, nezlob se!, Senet, Royal Game of Ur, Vrchcáby* apod. Tedy bude v něm relativně jednoduché tyto hry implementovat.

Framework vytvoří okno, kde bude hru zobrazovat. Bude podporovat vykreslování herní plochy a figurek hráčů. Obsluhovat uživatelský vstup – klávesnice a myš. Detekovat klikání na herní plochu (figurky). Bude podporovat střídání hráčů a umožní uživateli během hry dynamicky ovlivňovat její chod – přidávání a přesouvání figurek, přeskakování hráčů apod.

Použité knihovny:

`GLFW` – OpenGL knihovna (vytvoření okna = kontext, apod.)
`Glad` – načtení OpenGL (pointers to OpenGL functions)
`GLM` – matematika pro OpenGL
`STB` – načítání obrázků

1.2. Konečné řešení

Vytvořil jsem framework, který lze využít k implementaci výše zmíněných her a podobných dalších. Jako příklad jsem implementovat *The Royal Game of Ur*.

Framework je templatovaná třída, jejíž parametr ovlivňuje, jak se bude po spuštění chovat. Klíčovým typem je `EventSystem`, což je uživatelem definovaná třída, která je volána frameworkem a zároveň jí uživatel může využívat k ovlivňování běžící hry (viz dále).

Grafická stránka je řešená s pomocí OpenGL. Existuje podpora pro načítání několika textur. Ty mohou být rozděleny na ikony a ty aplikovány na herní figurky.

Hra vždy vykresluje herní plochu – s ní souvisí množina políček, na které mohou hráči během hry klikat. Mezi políčky mohou vést cesty, které mohou mít definované váhy.

Program jsem vyvíjel v prostředí *Visual Studio 2022* s verzí *C++20* a dokumentoval přes *GitLab*. Jedná se o cmake projekt, který jsem primárně vyvíjel na Windows 11, ale měl by být spustitelný i na Linux.

2. Uživatelská dokumentace

Tato část popisuje, jak `Framework` použít k vytvoření vlastní hry. V poslední podkapitole se zmíním o používání příkladové implementace *The Royal Game of Ur*.

```
TabletopFramework<FrameworkProperties> my_game(width, height,
   {
      FrameworkSettings(
       gameTitle,
       workingDir,
       minPlayers,
       maxPlayers)
   }
);
if(my_game.isValid())
   my_game.start();
```

Klíčovým typem je třída `TabletopFramework`. Ta potřebuje argument `FrameworkProperties`, což je uživatelem definovaný typ, ke kterému se dostaneme.

Její konstruktor vyžaduje několik argumentů –
width, height – šířka a výška hracího okna po spuštění (lze později změnit obvyklým způsobem)
`FrameworkSettings`, který vyžaduje -
gameTitle – řetězec pro název vytvořeného okna
workingDir – pracovní složka, vůči níž se vyhodnocují relativní cesty (např. pro textury)
minPlayers, *maxPlayers* – minimální a maximální počet hráčů

Konstruktor se pokusí inicializovat grafické rozhraní. K tomu musí načíst uživatelem specifikované textury a ikony (viz dále). Pokud při inicializaci dojde k nějakému problému, framework se dostane do FAIL stavu a nebude možné ho spustit.

Pokud je framework ve spustitelném stavu (lze zjistit skrze `isValid()`) lze jej spustit přes příkaz `start()`. Poté, co aplikace skončí, přejde do stavu DEAD a nebude možné jej spustit znovu.

2.1. Ovládání po spuštění

Ihned po spuštění se framework nachází ve stavu MENU. Zde je potřeba aby uživatel zadal aktuální počet hráčů do standartní konzole. Poté hra začne.

Po spuštění hra reaguje na uživatelský vstup. Postupně střídá, který hráč je na tahu. Hráč může klávesou „D“ „hodit kostkou“ (vygenerovat cenu) nebo „S“ ukončit tah.

Držením pravého tlačítka myši lze posouvat herní plán. Kolečkem lze přibližovat a oddalovat hru.

Klikáním na políčka lze vybírat figurky, které se na nich nachází. Takto lze vybrat pouze figurky, které patří aktuálnímu hráči. Vybraná figurka je označena červeným kruhem. Pokud hráč klikne mimo herní plán, figurka bude odznačena. Pokud klikne na políčko na herním

plánu, pokusí se zavolat Pathfinder, který pokud vrátí validní cestu, figurku na dané políčko přemístí a zaplatí „cenu“ cesty.

Hru lze také pozastavit klávesou „Space“. Pak nelze házet kostkou či střídat hráče. Vrátit se do hry lze opět klávesou „Space“. Alternativně lze klávesou „R“ hru resetovat (s aktuálním počtem hráčů) nebo „M“ vrátit se do menu, kde lze počet hráčů změnit.

Pokud hra skončí, musí hráč stisknout klávesu „R“ nebo „M“.

Kdykoliv je stisknuta klávesa „Escape“, je aplikace silou ukončena.

2.2. Specifikace framework

Nejdůležitější částí inicializace je typ `FrameworkProperties` (název není podstatný), který je předáván do framework. Musí splnit definovaný koncept `FrameworkConcept`. To znamená:

definovat následující strukturu typů:

```
FrameworkProperties
	EventSystem
	TextureLoader
		TextureAlias_type
	IconLoader
		IconAlias_type
	GameProperties
		PlayerProperties
		BoardProperties
			TileProperties
		PieceProperties
			PieceAlias
```

jednotlivé typy dále musí splňovat následující koncepty:

Nejprve zmíním `MinimalistIterator<T, R>`, který vyžaduje aby typ `T` obsahovat operator prefixové inkrementace (`++T`) jež vrací `T&`. Dále operátor dereference (`*T`), který vrací `R&`. A operátor pro nerovnost (`T != T`) vracející bool.

`TextureLoaderConcept` pro `TextureLoader`, tedy třída pro načítání textur. Ta musí definovat `TextureAlias_type`, který bude používán jako alias názvů textur. Dále metody `texturesBegin()` a `texturesEnd()`, které musí vracet hodnotu typu `T` splňující `MinimalistIterator<T, TextureEntry<TextureAlias_type>>`.

Tímto typem framework načítá po spuštění textury (iterací od `texturesBegin` k `texturesEnd`) a přiřazuje jim aliasy. Alias a cesta k textuře je předávána právě typem `TextureEntry`. Navíc pokud je `TextureAlias_type` označen jako const (např. `using TextureAlias_type = const std::string`) dojde při načítání textury ke zkopírování aliasu. Pokud je označen jako ne-const, dojde k vykradení dané `TextureEntry` instance.

Podobným způsobem funguje `IconLoaderConcept` pro `IconLoader`, který potřebuje typ `IconAlias_type` a funkce `iconsBegin()` a `iconsEnd()`. Ikony jsou definovány jako části textur. `IconEntry` potřebuje znát název ikony (`IconAlias_type`), název textury, pozici vzhledem

k levému hornímu rohu textury a rozměry ikony. Ikona je tedy obdélníková část dané textury.

`BoardProperties` představuje nastavení pro hrací plochu. Musí obsahovat dvě `constexpr float` hodnoty `WIDTH` a `HEIGHT`, definující šířku a výšku hrací plochy. Dále metodu `boardIconName()` která vrátí název ikony pro hrací plochu (ta musí být definovaná v skrze `IconLoader`).

`BoardLoaderConcept` pro `BoardLoader`, tedy typ, který načítá hrací plochu. Ten musí obsahovat metody `tilesBegin()` a `tilesEnd()` vracející `T` splňující `MinimalistIterator<T, TileEntry<TileProperties>>`. `TileEntry` zde definuje alias pro políčko na hracím plánu (alias je vždy x a y souřadnice jako celá čísla) a jeho reálnou světovou pozici vůči ikoně hrací plochy (od levého dolního rohu). Navíc musí obsahovat velikost políčka – buď lze každou specifikovat samostatně, nebo skrze `constexpr float` na `TileProperties` s názvem `TileSize`, který bude použit místo něj.

V celém frameworku pozice vždy reprezentuje střed daného elementu.

Dále metody `edgesBegin()` a `edgesEnd()` vracející `T` splňující `MinimalistIterator<T, EdgeEntry>`. `EdgeEntry` představuje hranu mezi dvěma políčky – jejich souřadnicemi – a obsahuje váhu této hrany.

Pokud `GameSettings` definuje `Pathfinder`, pak bude takto definovaný typ použit při hledání cest pro figurky ve hře (viz dále). Pokud nikoliv, bude použit výchozí `Pathfinder`.

Podobně, `BoardSettings` může definovat `EdgeValidator`, jehož fungování vysvětlím později. Opět pokud není definován, bude použit výchozí.

`PathfinderContext` pro `Pathfinder`, který musí obsahovat metodu `getPathBetween(from, to, context)` vracející `PiecePath`. *from* a *to* jsou typy `glm::ivec2` (celočíselné vektory) reprezentují políčka herního plánu a *context* je `PathfindingContext`, který dává k dispozici různé funkce pro zjištění aktuálního stavu hry (viz dále). `PiecePath` pak reprezentuje vektor políček, po kterých bude figurka postupovat.

`EdgeValidatorConcept` pro `EdgeValidator`, musí obsahovat konstruktor akceptující `GameContext<FrameworkProperties>` a metodu `isEdgeValid(edge)`, kde *edge* je `Edge`, tedy typ reprezentující hranu – obsahuje `glm::ivec2` *from*, *to* a cenu hrany `int`. O kontextu se zmíním dále.

2.3. Systém událostí (event system)

Konečně nejdůležitější částí je `EventSystemConcept`, který vyžaduje následující metody:

```
EventSystem(context);
```

Tedy konstruktor beroucí `GameContext<FrameworkContext>` (viz dále).

```
void init();
```

Zavoláno před prvním spuštěním hry a pouze jednou za celý běh aplikace. Slouží hlavně k vytvoření figurek pro všechny hráče (ale není to nutné provádět pouze zde).

Figurka je přiřazena danému hráči pod daným aliasem.

```
void turnBegin();
void turnEnd();
```

Volány na začátku a konci každého hráčského kola (pokud byla stisknuta klávesa „S“).

```
std::size_t gameBegin();
void gameOver();
```

Zavoláno na začátku hry – musí vrátit číslo hráče, který bude první na tahu – a na konci hry.

```
void restart();
```

Zavoláno pokaždé, když je hra restartována (včetně před prvním spuštěním).

Pokud hráč stiskne klávesu „R“, všechny herní figurky budou přesunuty na jejich výchozí pozice a poté bude zavolána metoda gameBegin().

```
void mousePressedAt(float, float);
void mousePressedAtTile(int, int);
```

Voláno po klinutí myši na libovolné pozici (předané `float` hodnoty reprezentují pozice ve světě beroucí v potaz přiblížení a posun hrací plochy – střed herní plochy je bodem [0, 0]), nebo po kliknutí na políčko na hracím plánu (předané int hodnoty jsou souřadnice daného políčka).

```
int generateCost();
```

Funkce zavolaná po stisknutí klávesy „D“. Vrátí novou hodnotu ceny pro aktuálního hráče, která bude přičtená k jeho dosavadní ceně. Ta se využívá při pohybu figurek. Implementace může být volání generátoru náhodných čísel s různými parametry. Je na uživateli, aby jí definoval.

```
bool isGameOver();
```

Volána opakovaně, pokud vrátí true ukončí hru a zavolá `gameOver()`.

```
bool canGenerateCost();
```

Volána po stisknutí klávesy „D“. Pokud vrátí true, zavolá `generateCost()`.

```
bool canSelectPiece();
```

Volána, pokud hráč klikne na políčko s figurkou. Pokud vrátí true, tato figurka bude vybrána. Vhodné, například pokud již hráč s některou figurkou pohnul a například pravidla nepovolují déle s jinou figurkou pohybovat.

```
bool canEndTurn();
```

Volána po stisknutí klávesy „S“. Pokud vrátí true bude aktuální kolo ukončeno, zavolána `turnEnd()`, vybrán další hráč a zavolána `turnBegin()`.

```
bool shouldProcessTilePress(int, int);
```

Volána po stisknutí na políčko na herním plánu. Pokud vrátí true, pokusí se vybrat figurku, pokud na daném políčku leží, nebo vybranou figurku na dané políčko přesunout (viz Pathfinding).

Vhodné například pokud některé políčko má mít speciální funkci (například přidat novou figurku).

```
bool passTile(pieceAlias, ivec2);
bool movePieceToTile(pieceAlias, ivec2);
```

Pokud dojde k nalezení cesty pro pohyb figurky, bude nejpre pro každé políčko na cestě zavolána `passTile()`, kde `pieceAlias` je alias figurky a `ivec2` je pozice políčka. Pokud kdykoliv vrátí false, bude pohyb ihned ukončen.

Nakonec zavolá `movePieceToTile()` se stejným aliasem a s políčkem, kam bude chtít figurku přesunout. Opět pokud vrátí false, bude pohyb přerušen.

Pokud pro všechna políčka vrátí true, dojde k pohybu a bude zavolána funkce `pieceMovedTo()`.

```
void pieceCreatedAt(playerId, pieceAlias, vec2);
void pieceCreatedAt(playerId, pieceAlias, ivec2);
void pieceMovedTo(playerId, pieceAlias, vec2);
void pieceMovedTo(playerId, pieceAlias, ivec2);
void pieceVisibilityChanged(playerId, pieceAlias, bool);
void pieceIconChanged(playerId, pieceAlias, iconAlias);
void pieceSizeChanged(playerId, pieceAlias, vec2);
```

Tyto metody jsou volány pokaždé, když figurka `pieceAlias`, hráče číslo `playerId`, byla postupně: vytvořena na pozici `vec2`, nebo políčku `ivec2`; přesunuta na pozici `vec2`, nebo políčko `ivec2`; změněna její viditelnost (true = viditelná); změněna její ikonka na `iconAlias`; nebo změněna její velikost na `vec2`.

2.4. Herní kontext (GameContext)

Uživatel může využívat `GameContext<FrameworkProperties>` instanci, kterou dostane při volání konstruktoru aby ovlivňoval dění ve hře.

Je potřeba ještě říct, že jednotlivé herní prvky obsahují vlastnosti (properties) definované uživatelem. Jedná se o hráče (`PlayerProperties`), políčka (`TileProperties`), figurky (`PieceProperties`).

```
std::size_t getCurrentPlayer() const;
std::size_t getCurrentCost() const;
void setCost(std::size_t newCost);
void setCurrentPlayer(std::size_t playerId);
```

Vrací nebo upravuje aktuálního hráče nebo jeho cenu.

```
auto& getEdgeValidator();
const auto& getEdgeValidator() const;
auto& getPathfinder();
const auto& getPathfinder() const;
```

Vrací reference na `EdgeValidator` a `Pathfinder` (viz dále). Užitečné např. pokud je potřeba je restartovat.

```
void createPiece(
   std::size_t playerID,
   const PieceAlias& pieceAlias,
   PieceProperties&& pieceProperties,
```

```
   glm::vec2 position,
   glm::vec2 size,
   const IconAliasRaw& iconName);

void createPieceAtTile(
   std::size_t playerID,
   const PieceAlias& pieceAlias,
   PieceProperties&& pieceProperties,

   int x, int y,
   glm::vec2 size,
   const IconAliasRaw& iconName);
```

Vytvoří novou figurku pro daného hráče, s daným aliasem a vlastnostmi. Vytvoří jí buď na libovolné pozici nebo na herním políčku s danou velikostí a ikonou.

```
void movePiece(
   std::size_t playerID,
   const PieceAlias& pieceAlias,

   const glm::vec2& position);

void movePieceToTile(
   std::size_t playerID,
   const PieceAlias& pieceAlias,

   int x, int y);
```

Přesune existující figurku daného hráče a aliasu na novou libovolnou pozici nebo na herní políčko.

```
void setPieceVisibility(
   std::size_t playerID,
   const PieceAlias& pieceAlias,

   bool visible);
```

Změní viditelnost dané figurky.

```
void setPieceIcon(
   std::size_t playerID,
   const PieceAlias& pieceAlias,
   const IconAliasRaw& icon);
```

Změní ikonu dané figurky na novou.

```
bool anyPieceOnTile(const glm::ivec2& tile) const;
```

Vrátí true, pokud se na daném políčku nachází figurka libovolného hráče.

```
bool playerHasPieceOnTile(std::size_t playerId, const glm::ivec2& tile) const;
```

Vrátí true, pokud se na daném políčku nachází figurka daného hráče.

```
const PieceAlias& getPieceAt(std::size_t playerId, const glm::ivec2& tile);
```

Vrátí alias figurky daného hráče na daném políčku, pokud se na něm nachází (jinak selže).

```
bool isPieceAtTile(std::size_t playerId, const PieceAlias& piece);
```

Vrátí true pokud daná figurka je na některém políčku.

```
glm::ivec2 getTilePosition(std::size_t playerId, const PieceAlias& piece);
```

Vrátí pozici políčka na kterém se daná figurka nachází.

```
bool isPieceSelected();
```

Vrátí true, pokud je některá figurka vybrána.

```
bool selectPiece(std::size_t playerId, const PieceAlias& piece);
```

Vybere danou figurku.

```
void deselectPiece();
```

Zruší výběr figurky.

```
const glm::vec2& getPiecePosition(std::size_t playerId, const PieceAlias& piece)
const
```

Vrátí pozici figurky.

```
const glm::vec2& getPieceSize(std::size_t playerId, const PieceAlias& piece) const
```

Vrátí velikost figurky.

```
PlayerProperties& getPlayerProperties(std::size_t playerId)
const PlayerProperties& getPlayerProperties(std::size_t playerId) const
```

Vrátí vlastnosti pro daného hráče.

```
TileProperties& getTileProperties(const glm::ivec2& tile);
Const TileProperties& getTileProperties(const glm::ivec2& tile) const;
```

Vrátí vlastnosti daného políčka.

```
PieceProperties& getPieceProperties(std::size_t playerId, const PieceAlias&
alias);
const PieceProperties& getPieceProperties(std::size_t playerId, const PieceAlias&
alias) const;
```

Vrátí vlastnosti dané figurky.

```
auto tilePropertiesBegin();
auto tilePropertiesEnd();

auto tilePropertiesBegin() const;
auto tilePropertiesEnd() const;

auto piecePropertiesBegin(std::size_t playerId);
auto piecePropertiesEnd(std::size_t playerId);

auto piecesBegin(std::size_t playerId) const;
auto piecesEnd(std::size_t playerId) const;
```

Vrací iterátory postupně přes všechny vlastnosti políček; všechny vlastnosti figurek a všechny aliasy figurek.

2.5. Hledání cest (pathfinding)

Pathfinding, tedy hledání cesty pro figurku, se provádí, pokud hráč figurku vybral a stisknul některé další políčko. Pak je zavolaná na instanci `Pathfinder` (buď uživatelem definované nebo výchozí) metoda `getPathBetween(…)`.

Ta má za úkol vrátit reprezentaci cesty (tedy vektor souřadnic políček). Pokud je prázdná, cesta neexistuje. Jinak dojde k jejímu provedení (viz `passTile()` a `movePieceToTile()`).

Má k dispozici `PathfinderContext<FrameworkProperties>`, který definuje následující metody:

```
std::size_t getCurrentPlayer() const;
std::size_t getCost() const;
```

Vrátí aktuálního hráče a jeho cenu.

```
int getEdgeCost(const glm::ivec2& from, const glm::ivec2& to);
```

Vrátí cenu dané hrany (vedoucí z *from* do *to*)

```
bool anyPieceOnTile(const glm::ivec2& tile) const;
bool playerHasPieceOnTile(std::size_t playerId, const glm::ivec2& tile) const;
```

Vrátí true, pokud se na daném políčku nachází libovolná figurka; figurka daného hráče.

```
auto edgesFromBegin(const glm::ivec2 from) const;
auto edgesFromEnd(const glm::ivec2 from) const;
```

Vrátí iterátory pro všechny hrany vycházející z daného políčka.

```
bool isValidEdgeBetween(const glm::ivec2& from, const glm::ivec2& to);
```

Tato metoda volá `EdgeValidator`, který má přístup k celému hernímu kontextu skrze `GameContext`. Je tedy schopný odpovědět, zda daná hrana je validní v aktuálním stavu (např. pro aktuálního hráče). Protože se jedná o instance, mohou obsahovat složitější logiku skrze privátní proměnně.

Výchozí `Pathfinder` se pouze dotáže, zda existuje hrana mezi danými políčky a pokud si jí hráč může dovolit.

Již zmínění `EdgeValidator` dostane `GameContext<FrameworkProperties>` při volání jeho konstruktoru. Musí definovat metodu:

```
bool isEdgeValid(const Edge& edge) const;
```

Výchozí implementace pro každou hranu vrací true.

Použítí `EdgeValidatoru` může výrazně zjednodušit implementaci `Pathfinderu`.

2.6. The Royal Game of Ur

Příkladová implementace této hry využívá většiny výše zmíněných elementů. Její pravidla lze vyhledat např. na internetu a nebudu se zde jimi příliš zabývat.

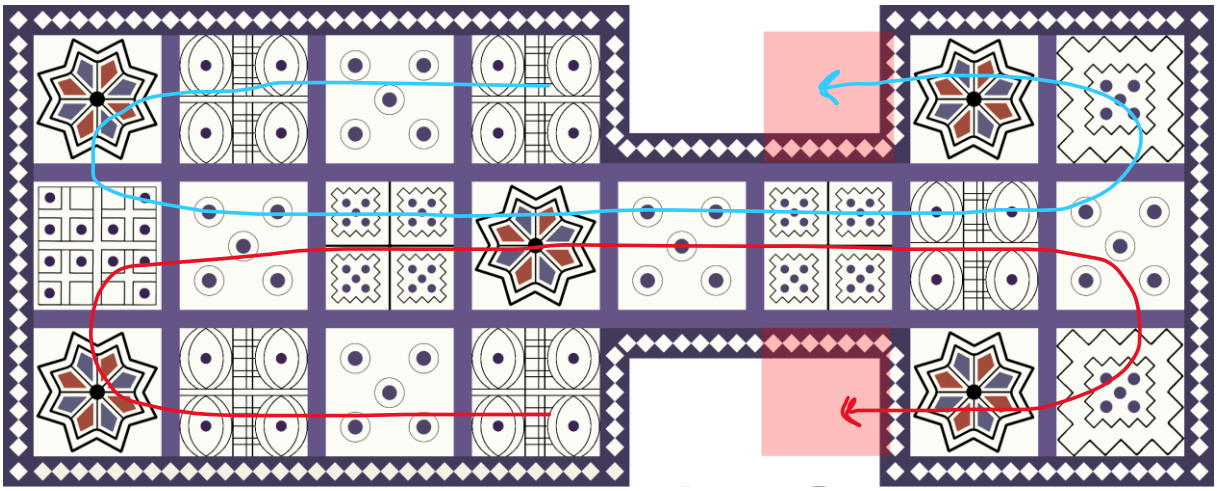
Hru musí vždy hrát 2 hráči (viz MENU). Hráč 0 hraje bílými figurkami, hráč 1 černými.

Na začátku kola musí hráč hodit kostkami (výsledky 0-4). Pokud nehodil 0, musí položit jednu ze svých figurek na herní plán. To udělá stisknutím na jedno z prvních čtyř políček (podle toho, kolik hodil).

Pokud kdykoliv vstoupí na políčko s rozetou, hází znovu. Dokud hráč může házet, nebo pohybovat s figurkami, nemůže kolo ukončit.

Jakmile jsou na hrací ploše nějaké figurky, po hození se místo přidání nové figurky, může hráč rozhodnout pro pohyb již vyložené. Nikdy nelze aby na jednom políčku existovali dvě figurky. Pokud hráč chce vstoupit na políčko s nepřátelskou figurkou, bude tato figurka z herního plánu odstraněna. To lze pokud, se nenacházela na rozetě.

Důležité je, že vždy se při pohybu musí zcela spotřebovat cena. Tedy pohyb musí obsahovat tolik políček, jako je aktuální cena.



Cílem je dostat všechny své figurky přes herní plán, kde uprostřed se mohou nacházet figurky obou hráčů. Aby se figurka dostala z herního plánu, musí hráč stisknout na políčko mimo herní plochu (na obrázku červená barva) a opět mít přesnou cenu. Cesta jednotlivých hráčů je vyznačena modrou a červenou čarou. Rozeta je osmi-cípa hvězda, která je na plánku pětkrát.

První hráč, který dostane všech 7 svých figurek na konec, zvítězí. Při opakovaných hrách si hra pamatuje kolikrát, který hráč zvítězil.

3. Programátorská dokumentace

V této části zmíním hlavní elementy framework a nastíním, jak fungují.

3.1. OpenGL

Grafické rozhraní je řešeno skrze `OpenGL`. V aplikaci je reprezentováno třídou `RenderAPI`. Do frameworku by bylo možné v budoucnu přidat jiná grafická rozhraní (`DirectX` apod.) a zachovat strukturu `RenderAPI`. Proto mají konkrétní implementace některých typů přízvisko `OpenGL`.

`RenderAPI` má za úkol vytvořit vrstvu mezi low-level voláním `OpenGL` funkcí a frameworkem. Umí vytvořit okno aplikace. Spravuje jednotlivé buffer objekty. Obsahuje callback funkce pro změnu velikosti okna a zaznamenání uživatelského vstupu. Dokáže načítat textury ze souborů a vytvářet shadery.

Konečně obsahuje hlavní vykreslovací smyčku, která volá jednolivé renderPasses – tedy funkce definované jinými třídami.

`GameGraphicsFacade` je další vrstvou mezi `RenderAPI` a frameworkem. Konkrétně mezi `GameManager` třídou (viz dále). Ten ví, že ve frameworku existují objekty – dokáže je vytvářet – a umí je zobrazit přes RenderAPI. Definuje renderPass, který vykresluje herní objekty.

Na začátku načte všechny ikony.

Objekt má nějakou pozici ve světe, velikost a ikonu. Objekty jsou vykreslované jako obdélníky (skrze instancové kreslení) a na ně je aplikovaná textura. To, která textura, je dáno podle ikony (její textura) a st (uv) souřadnice jsou spočtené z pozice a rozměrů dané ikony.

Kvůli tomu jsou objekty umístěny do `ObjectBufferu`, který dokáže držet předem dané množství objektů. Pro každý další objekt je potřeba vytvořit další `ObjectBuffer` atd. Všechny objekty ve stejném bufferu mají ikony ze stejné textury, aby bylo vykreslování efektivní. Proto je ideální, když uživatel použije pouze malé množství textur s hodně ikonama (atlas).

`Facade` dále spravuje kameru, kterou může uživatel posouvat. Objekty nelze mazat ani skrývat. S kamerou lze ale pouze pohybovat ve dvou dimenzích a je tedy možné objekty přesunout tak, aby nebyly vidět (za kameru). Používá se ortografické projekce.

Využívá se dvou hlavních shaderů. Hlavní vykresluje většinu herních elementů, jak bylo popsáno výše, jako čtverce s texturou. Druhý vykresluje zvýraznění kolem vybrané figurky jako pulsující kružnici.

Cílem `Facade` je dát `GameManageru` schopnost vytvářet tyto objekty, a případně je za běhu upravovat.

3.2. Game Manager

Herní logiku obstarává `GameManager`. Obsahuje herní prvky jako hráče, kteří drží figurky. To jsou v podstatě objekty získané z `Facade` a dokážou tak provádět různé užitečné funkce, jako přesouvat se, skrýt nebo odhalit se, měnit své ikony atd.

Ví, který hráč je zrovna na tahu a jakou má cenu. Většina vstupů od uživatele je spojena s aktuálním hráčem. Na konci tahu je id hráče zvýšeno o jedna. Pře překonání aktuálního počtu hráčů se vrátí zpět k 0.

Obsahuje herní plochu skrze typ `PlayingBoard`. Ten načte od uživatele všechna políčka a hrany mezi nimi.

Figurky se mohou nacházet na políčkách. Figurka nemůže být na více políčkách najednou, ale jedno políčko může obsahovat více figurek (je na uživateli, zda toto chce či ne). Figurka, která je na políčku a patří aktuálnímu hráči, může být vybrána. Poté může být přesunuta na jiné políčko, pokud `Pathfinder` najde vhodnou cestu (viz Pathfinding).

Klikání na políčka se detekuje zde. Jednotlivá políčka jsou obdélníky definované pozicí a velikostí. Při kliknutí myši se vyhledá, zda existuje políčko, na které mohlo být kliknuto. Není definované, které políčko se vyhodnotí, pokud se překrývají.

`GameManager` a `PlayingBoard` slouží i jako rozhraní pro `GameContext`, tak aby mohli být jednotlivé funkce prováděny.

3.3. Tabletop Framework

Hlavním typem, který uživatel využívá je `TabletopFramework`. Ten byl již několikrát zmíněn. Obsahuje stavový automat. Překládá jednotlivé příkazy od uživatele do `GameManageru`. Obsahuje hlavní herní smyčku (svázanou s vykreslovací).

Jednotlivé stavy jsou:

`INIT`

Výchozí stav, v konstruktoru dojde k inicializaci. Pokud selže, přejde do stavu FAIL. Jinak přejde do stavu MENU, ale před tím zavolá `init()` na `EventSystemu`. V tuto chvíli ho lze spustit příkazem `start()`.

`start()` spustí hlavní herní smyčku, pokud není ve stavu FAIL nebo DEAD.

`MENU`

Zde očekává vstup od uživatele – kolik hráčů bude hrát. Dokud nedostane platný vstup, číslo mezi min a max hráči, bude se ptát znovu. Jakmile dostane platné číslo, přesune se do stavu RUNNING.

`RUNNING`

Zde je volán `GameManager.tick()` a hra běží. Je možné interagovat s figurkami. Pokud je hra pozastavena stisknutím „Space“ přejde do stavu PAUSED.

Také dochází ke kontrole `isGameOver()`. Pokud vrátí true, přejde do stavu OVER.

`PAUSED`

Zde nedochází k volání `GameManager.tick()` a nelze provádět většinu akcí. Lze stisknout klávesu „Space“ a vrátit se do stavu RUNNING. Nebo klávesami „R“ hru resetovat, či „M“ se vrátit do stauv „MENU“.

`OVER`

Podobný stav jako PAUSED, ze kterého se ale nelze vrátit přes „Space“.

```
DEAD
```

Stav, do kterého se dostane, pokud je hra ukončena („Esc“ či zavření okna).

```
FAIL
```

Stav, do kterého se dostane, pokud kdykoliv dojde ve frameworku při inicializaci k chybě.

4. Možná rozšíření

Do frameworku by šlo přidat další funkce, které jsem už z časových důvodů dále neimplementoval. Šlo by je přidat někdy v budoucnu jako možná rozšíření.

Házení kostkami – většina her obsahuje klasické kostky, případně házení s několika různými kostkami. Mohlo by se jednat o třídu, kterou by uživatel mohl využít v `EventSystemu`.

Dijkstra pathfinder – hledání nejkratší cesty, brát v potaz cykly a váhy hran. Zde je problém v tom, že každá hra obsahuje jinou topologii a nemusí být úplně jednoduché implementovat univerzální systém, který to zvládne obsloužit. Navíc hodně her má složitá pravidla o tom, kdy lze s figurkou pohnout a ne – ne nutně najít nejkratší cestu.

Výchozí načítání textur, ikon, hrací plochy případně i figurek při inicializaci. Zde je hlavní problém v tom, že si uživatel může definovat vlastní typy pro jednotlivé aliasy. Načítání například z textových souborů by tak bylo velmi komplikované. Proto mi přijde aktuální způsob přes iterátory jako dobrý kompromis.

Více nastavení – barvu zvýraznění a zda se má provádět, zda pohybovat figurky „drag and drop“ způsobem, zvýrazňovat je při najetí myši apod.

Detekovat klikání na všechny figurky. Podporu pro více figurek stejného hráče na jednom políčku (například pokládání do kruhu).

Textové UI – tlačítka a panely. Zde jsem uvažoval buď o vlastní implementaci buď skrze font-texturu nebo knihovnu `FreeType`.

5. Závěr

Cílem bylo vytvořit framework pro tvorbu některých deskových her. Řekl bych, že toho jsem docílil a že vytvořit v něm hru *The Royal Game of Ur*, nebyl velký problém.